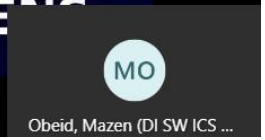
# Session #2






SIEME

Obeid, Mazen (DI SW ICS MNA RD QA QSIM 1)



## VHDL Packages

- A **PACKAGE** is a design unit to make **TYPE**, **COMPONENT**, **FUNCTION**, and other declarations visible to other design units.
- A **PACKAGE** consists of:
  1. a **PACKAGE DECLARATION** that is used to declare **TYPES**, **COMPONENTS**, **CONSTANTS**, **FUNCTIONS (header)**, **PROCEDURES (header)**, etc.
  2. an optional **PACKAGE BODY**, where **PROCEDURES (body)** and **FUNCTIONS (body)** are defined.
- A package is made visible to an entity with a **USE** clause.

- ✓ In VHDL, data types are usually declared in a *package*. A package is a place for declaring resources that are common to several *design entities*.
- ✓ The use clause at the top of FILTER (use WORK.TYPES.all;) allows the names declared in the package to be used directly within the entity. The first part of the name (WORK) is the name of a library, the second part (TYPES) is the name of a package on that library, and the final part (all) gives visibility of all the names declared in that package.

```
package TYPES is
  type CODE is (A, B, C, D, E);
end TYPES;
```

```
use WORK.TYPES.all;
entity FILTER is
  port (IP: in  CODE;
        OP: out CODE);
end FILTER;
```

```
use WORK.TYPES.all;
...
signal I, O: CODE;
...
U1: FILTER port map (IP => I,
                     OP => O);
```



SIEMENS

## VHDL Packages

- A **PACKAGE** is a design unit to make **TYPE**, **COMPONENT**, **FUNCTION**, and other declarations visible to other design units.
- A **PACKAGE** consists of:
  1. a **PACKAGE DECLARATION** that is used to declare **TYPES**, **COMPONENTS**, **CONSTANTS**, **FUNCTIONS (header)**, **PROCEDURES (header)**, etc.
  2. an optional **PACKAGE BODY**, where **PROCEDURES (body)** and **FUNCTIONS (body)** are defined.
- A package is made visible to an entity with a **USE** clause.

```
PACKAGE andPackage IS
   COMPONENT and2 IS
      PORT( a, b:  IN bit; c:  OUT bit);
   END COMPONENT and2;
END PACKAGE andPackage;
USE WORK.andPackage.ALL;
ENTITY circuit IS
   PORT( a, b, x, y:  IN bit;
         d:  OUT bit);
END ENTITY circuit;
ARCHITECTURE mixed OF circuit IS
   FOR gate:  and2 USE ENTITY WORK.and2 (and2);
   SIGNAL c, z:  bit;
BEGIN
   gate:  and2 PORT MAP (a, b, c);
   d <= c XOR z;
   op:  PROCESS (x, y) IS BEGIN
      z <= x OR y;
   END PROCESS op;
END ARCHITECTURE mixed;
```



14:02
SIEM
MO
Obeid, Mazen (DI SW ICS ...
Obeid, Mazen (DI SW ICS MNA RD QA QSIM 1)

# Clocks & Registers : Rising Edge

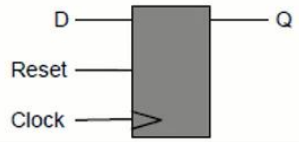
## RISING_EDGE

- **RISING_EDGE is TRUE when Clock changes from '0' or 'L' to '1' or 'H'**

```
process
begin
  wait until RISING_EDGE(Clock);

  if Reset = '1' then
    Q <= '0';
  else
    Q <= D;
  end if;
end process;
```

synchronous reset




## if RISING_EDGE

```
process (Clock)
begin
  if RISING_EDGE(Clock) then
    Q1 <= D;
  end if;
end process;
```

```
process (Clock)
begin
  if FALLING_EDGE(Clock) then
    Q2 <= D;
  end if;
end process;
```



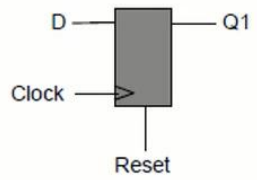
SIEMENS

Obeid, Mazen (DI SW ICS MNA RD QA QSIM 1)

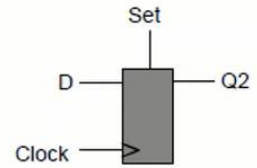
## Clocks & Registers : Async reset/set

**Asynchronous Set or Reset**

```
process (Reset, Clock)
begin
  if Reset = '1' then
    Q1 <= '0';
  elsif RISING_EDGE(Clock) then
    Q1 <= D;
  end if;
end process;
```




```
process (Set, Clock)
begin
  if Set = '1' then
    Q2 <= '1';
  elsif RISING_EDGE(Clock) then
    Q2 <= D;
  end if;
end process;
```




- The sensitivity list must contain the clock, the asynchronous set or reset, and nothing else



18:31

SIEMENS

Obeid, Mazen (DI SW ICS MNA RD QA QSIM 1)

MO

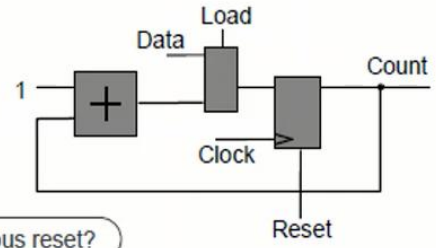
# Clocks & Registers : Async Vs. Sync

## Synchronous Vs. Asynchronous Actions

```
process (Clock, Reset)
begin
  if Reset = '1' then
    Count <= "00000000";
  elsif Rising_edge(Clock) then
    if Load = '1' then
      Count <= UNSIGNED(Data);
    else
      Count <= Count + 1;
    end if;
  end if;
end process;
```

Asynchronous reset

Synchronous load

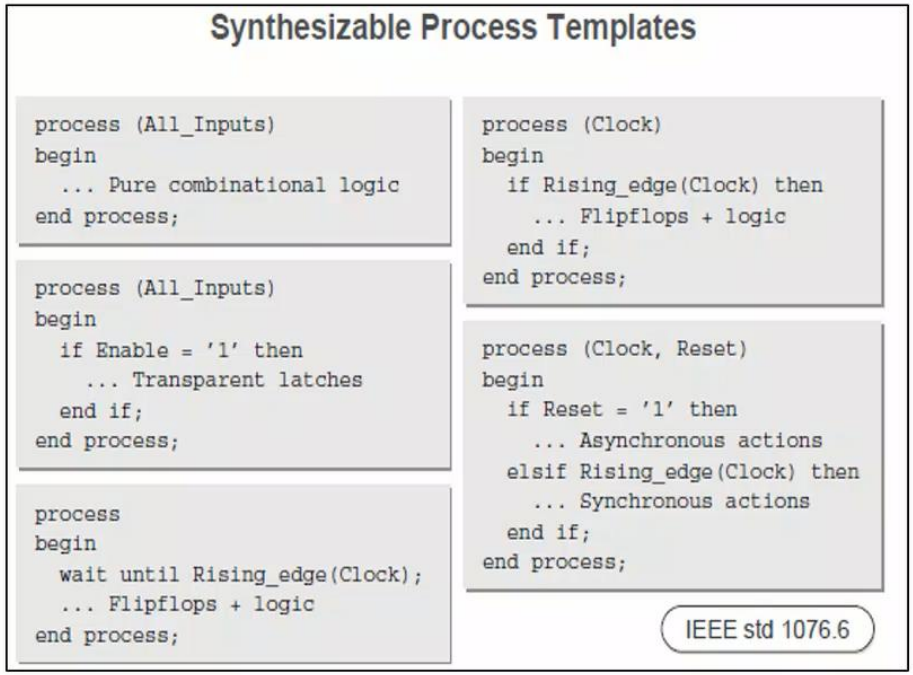



What if the device has no asynchronous reset?



SIEMENS

# Synthesizable Process Templates




- For pure combinational logic, 3 rules must be obeyed. The sensitivity list must be complete (i.e. contain all signals read by the process), there must be no incomplete assignments (i.e. all outputs must be assigned for all input conditions), and there must be no feedback.
- For transparent latches, the process should contain just a single if statement, which may itself contain anything permitted in a combinational process. Like a combinational process, the sensitivity list must be complete, and there must be no feedback.
- For flip-flops, the process must be synchronized to a clock using one of the latter 3 styles shown opposite. A clocked process must include the clock and the asynchronous reset (or set), if there is one. The VHDL statements thus synchronized to a clock may include anything permitted in a combinational process.
- Other styles of process are permitted by certain synthesis tools but are not generally recommended. Examples include statements before or after the if statement in a clocked process, and multiple wait statements.

19:41



SIEMENS

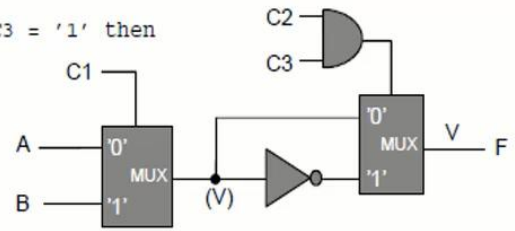
Obeid, Mazen (DI SW ICS MNA RD QA QSIM 1)

MO

# Sequential statements : If statements

**If Statements**

```
process (C1, C2, C3, A, B)
  variable V: STD_LOGIC;
begin
  if C1 = '0' then
    V := A;
  else
    V := B;
  end if;
  if C2 = '1' and C3 = '1' then
    V := not V;
  end if;
  F <= V;
end process;
```




An *if statement* is a sequential statement that conditionally executes others sequential statements, depending upon the value of some condition. An *if statement* may optionally contain an *else* part, executed if the condition is false.

***end if***

Every *if statement* must have a corresponding *end if*.

Note that *end if* is two keywords - writing *endif* is a common mistake!

***Synthesis***

If statements are synthesized by generating a multiplexer for each signal and variable assigned within the if statement.



21:22

SIEMENS

## Sequential statements : Nested If statements

**Multiple and Nested Statements**

```
process (S, T, A, B, C, D)
begin
  if S = '1' then
    F <= '0';
    G <= '0';
    if T = '1' then
      F <= A and C;
      G <= B or D;
    end if;
  else
    F <= A or B;
    G <= C and D;
  end if;
end process;
```

Complete sensitivity list

- You can write any number of sequential statements within each branch of an if statement, and you can nest one *if* statement inside another as shown here.
- If the process is describing combinational logic, then it is important that you include all inputs to the process in the sensitivity list at the top.



24:55

SIEMENS

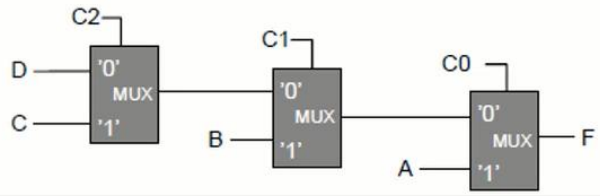
Obeid, Mazen (DI SW ICS MNA RD QA QSIM 1)

## Sequential statements : elsif statements

**Elsif**

```
process (C0, C1, C2, A, B, C, D)
begin
  if C0 = '1' then
    F <= A;
  elsif C1 = '1' then
    F <= B;
  elsif C2 = '1' then
    F <= C;
  else
    F <= D;
  end if;
end process;
```

- Priority implies multiple logic levels (before optimization)




- Testing a series of conditions is made easier by the *elsif* part of the *if* statement.

***Priority***

The use of *elsif* and the use of nested *if* statements both imply priority; the first condition takes priority over the second condition.



26:25

SIEM

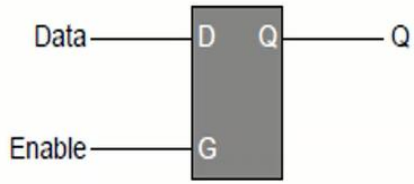
MO

Obeid, Mazen (DI SW ICS ...

Obeid, Mazen (DI SW ICS MNA RD QA QSIM 1)

## Sequential statements : Incomplete assignments

```
process (Enable, Data)
begin
  if Enable = '1' then
    Q <= Data;
  end if;
end process;
```




- Incomplete assignment infers a transparent latch

- If a signal is only assigned under certain input conditions, then it will retain its old value under other input conditions



27:53

SIEM

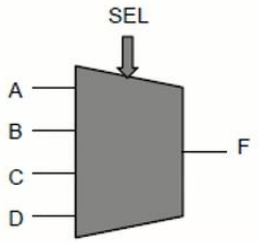
MO

Obeid, Mazen (DI SW ICS ...

Obeid, Mazen (DI SW ICS MNA RD QA QSIM 1)

## Sequential statements : Case statements

```
case SEL is
  when "00" =>
    F <= A;
  when "01" =>
    F <= B;
  when "10" =>
    F <= C;
  when "11" =>
    F <= D;
  when others =>
    F <= 'X';
end case;
```




- The case statement is an efficient way of executing a multi-way branch. The case statement lists all possible values of the expression at the top of the statement (case SEL is...).
- Below example shows some more features of the case statement

```
process (ADDRESS)
begin
  A <= '0';
  B <= '0';
  case ADDRESS is
    when 0 to 7 =>
      A <= '1';
    when 8 to 15 =>
      B <= '1';
    when 16 | 20 | 24 | 28 =>
      A <= '1';
      B <= '1';
    when others =>
      null;
  end case;
end process;
```

A range of integer values

Alternatives

Are transparent latches inferred?

33:35



SIEMENS

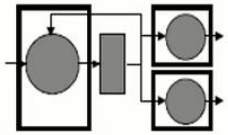
# Conditional Signal Assignment

**Conditional Signal Assignments**

```
architecture Concurrent of FSM is
  type StateType is (Idle, Go1, Go2);
  signal State, NextState: StateType;
begin
  Registers: process (Clock, Reset)
  begin
    if Reset = '1' then
      State <= Idle;
    elsif Rising_edge(Clock) then
      State <= NextState;
    end if;
  end process;

  F <= '1' when State = Go1 else '0';
  G <= '1' when State = Go2 else '0';

  NextState <= Go1 when State = Idle and Start = '1'
          else Go2 when State = Go1
          else Idle;
end;
```



- ✓ A conditional signal assignment is a concurrent statement that selects the expression to be assigned by evaluating a series of conditions.
- ✓ The conditions are evaluated in turn until one evaluates to true, then the corresponding expression is selected

- **Conditional signal assignment**

```
F <= '1' when State = Go1 else '0';
```

- **Equivalent processes**

```
process (State)
begin
  if State = Go1 then
    F <= '1';
  else
    F <= '0';
  end if;
end process;
```

```
process
begin
  if State = Go1 then
    F <= '1';
  else
    F <= '0';
  end if;
  wait on State;
end process;
```



SIEMENS

# Selected Signal Assignment

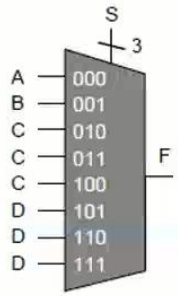
## Selected Signal Assignments

- **Selected signal assignment**

```
Mux: with S select
  F <= A when "000",
       B when "001",
       C when "010" | "011" | "100",
       D when others;
```

- **Equivalent processes**

```
Mux: process (S, A, B, C, D)
begin
  case S is
  when "000" =>  F <= A;
  when "001" =>  F <= B;
  when "010" | "011" | "100" => F <= C;
  when others => F <= D;
  end case;
end process;
```




- ✓ The selected signal assignment is a concurrent statement equivalent to a process containing a case statement. For example, the assignment labelled Mux opposite is equivalent to the process labelled Mux.
- ✓ Note that the selected statement can contain multiple choices and an others choice, exactly like the case statement. Also, because concurrent assignments are equivalent to processes, they may be labelled

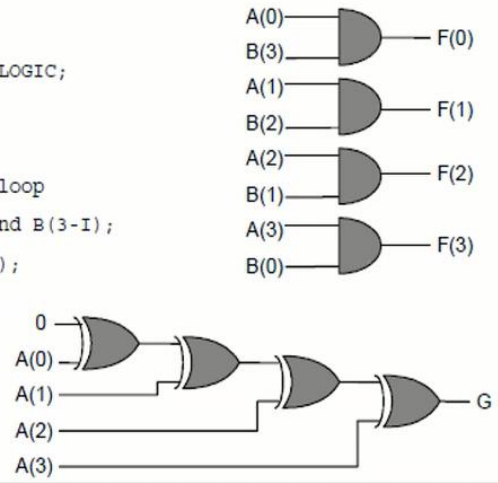


40:42
SIEM
MO
Obeid, Mazen (DI SW ICS ...
Obeid, Mazen (DI SW ICS MNA RD QA QSIM 1)

Press Esc to exit full screen

# Sequential statements : For Loops

```
process (A, B)
  variable V: STD_LOGIC;
begin
  V := '0';
  for I in 0 to 3 loop
    F(I) <= A(I) and B(3-I);
    V := V xor A(I);
  end loop;
  G <= V;
end process;
```




- ✓ *For loops* are sequential statements that are often used in conjunction with vectors. The statements inside the loop are repeated for each value in the range given at the top of the loop.
- ✓ In the example opposite, the loop is executed 4 times with the loop parameter I taking the values 0, 1, 2 and 3 respectively. The range at the top of the loop may be ascending (E.g. 1 to N) or descending (E.g. N downto 1).
- ✓ There are **two important rules governing for loops**.
  - ✓ Firstly, the value of the loop parameter can be read inside the loop, but must not be assigned, so you cannot alter the execution of a loop by changing the value of the loop parameter inside the loop!
  - ✓ Secondly, the loop parameter cannot be used outside the loop. The loop statement actually serves as a declaration of the loop parameter (as a constant).
- ✓ ***For loops* are synthesized to repeated hardware structures**, providing the bounds of the loop are fixed. In the example opposite, the first assignment in the loop would result in the synthesis of 4 AND gates. The second assignment synthesized to a cascaded structure of XOR gates



SIEMENS

## Sequential statements : Other Loops

```
for PARAMETER in LOOP_RANGE loop
  ...
end loop;
```

```
while CONDITION loop
  ...
end loop;
```

```
LOOP_LABEL: loop
  ...
  exit;
  ...
  exit LOOP_LABEL;
  ...
  next LOOP_LABEL;
  ...
end loop LOOP_LABEL;
```

Jump out of loop

Jump back to top



SIEMENS

# VHDL Generate Statements

```
architecture A1 of ENT is
begin
  G1: for I in SOME_RANGE generate
    -- component instances and processes
  end generate;

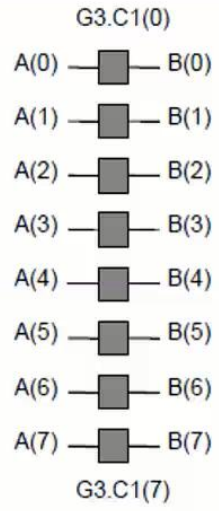
  G2: if CONDITION generate
    -- component instances and processes
  end generate G2;

  G3: for I in 0 to 7 generate
    C1: COMP port map (A(I), B(I));
  end generate G3;

end A1;
```

Regular structure

Optional structure




- ✓ Regular structures can be described in VHDL using the generate statement. **They avoid writing large numbers of very similar concurrent statements,** E.g., component instantiation statements.
- ✓ Generate statements can also be used to describe design variants, often in conjunction with generic parameters.
- ✓ **Generate statements can be nested indefinitely and** can be used in conjunction with any other concurrent statement.
- ✓ The **loop range** and the if condition within the generate statement must be **globally static**, i.e.. they may depend on generic constants.
- ✓ **Generate statements must be labelled**.
- ✓ **Generate statements should not be used in place of loop statements for synthesizing regular structures** such as shift registers, because being concurrent statements, they carry a significant simulation overhead when compared to sequential statements (within a process).
- ✓ **Generate statements should be reserved for repeating large blocks of logic**

57:54



SIEMENS

# More on Data Types : Subtypes

## Array Subtypes

- Naming an array subtype avoids repeating the index constraint

```
library IEEE;
use IEEE.STD_LOGIC_1164.all;

package BusTypes is
  subtype DataBus is STD_LOGIC_VECTOR(7 downto 0);
end;
```

```
library IEEE;
use IEEE.STD_LOGIC_1164.all, WORK.BusTypes.all;

entity CNTL is
  port (CLK:  in  STD_LOGIC;
        D:    in  DataBus;
        Sin:  in  STD_LOGIC_VECTOR(3 downto 0);
        Q:    out DataBus;
        Sout: out STD_LOGIC_VECTOR(1 downto 0));
end;
```

```
PACKAGE new_types IS
   SUBTYPE eight_bit IS integer RANGE -128 TO 127;
END PACKAGE new_types;
```

- ✓ **Subtypes are Type derivatives**
  - ✓ Usually, we use subtypes to constrain or limit a specific type {either use defined or pre-defined} to a specific usage.
  - ✓ **Subtype** defines a restricted set of values of a **base type**.

- ✓ We have used the type STD_LOGIC_VECTOR to represent a bus or vector, giving the width of the vector using an index constraint (e.g. STD_LOGIC_VECTOR(7 downto 0)). Instead of repeating the index constraint each time a new signal or variable is declared, we can declare a named subtype as shown in the example opposite.



01:28:51
SIEM
MO
Obeid, Mazen (DI SW ICS ...
Obeid, Mazen (DI SW ICS MNA RD QA QSIM 1)

# More on Data Types : Array & Types Attributes

## Array and Type Attributes

```
signal  A: STD_LOGIC_VECTOR(7 downto 0);
subtype SHORT is INTEGER range 0 to 15;
type    MODE  is (W, X, Y, Z);
```

- **Array attributes (use whenever possible)**

```
A'LOW   = 0
A'HIGH  = 7
A'LEFT  = 7
A'RIGHT = 0
```

```
A'RANGE = 7 downto 0
A'REVERSE_RANGE = 0 to 7
A'LENGTH = 8
```

- **Type attributes (avoid for synthesis)**

```
SHORT'LOW   = 0
SHORT'HIGH  = 15
SHORT'LEFT  = 0
SHORT'RIGHT = 15
```

```
MODE'LOW   = W
MODE'HIGH  = Z
MODE'LEFT  = W
MODE'RIGHT = Z
```

- ✓ Type attributes and array attributes allow properties of a type to be accessed in an explicit way. The advantage of using these attributes is that any change to the definition of the type is automatically reflected in the values of the attributes, making the VHDL text easier to maintain.
- ✓ Array attributes should be used whenever possible. They make the code more readable and maintainable

01:31:38

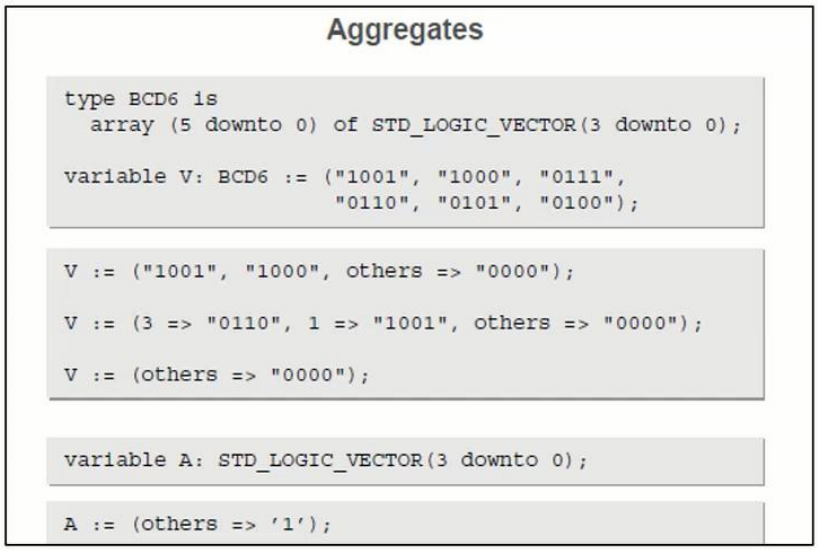


SIEM

MO
Obeid, Mazen (DI SW ICS ...

Obeid, Mazen (DI SW ICS MNA RD QA QSIM 1)

# Aggregates




- ✓ Sometimes we need to write the value of a complete array, rather than assigning individual elements of the array. One case where this is necessary is the default value in an object declaration.
- ✓ An aggregate is used to construct the entire composite value from the values of the individual elements.
- ✓ **An aggregate is a list of expressions, separated by commas and enclosed in brackets**. The expressions in an aggregate can include operators and even other aggregates, so long as they yield a value of the element type of the array.

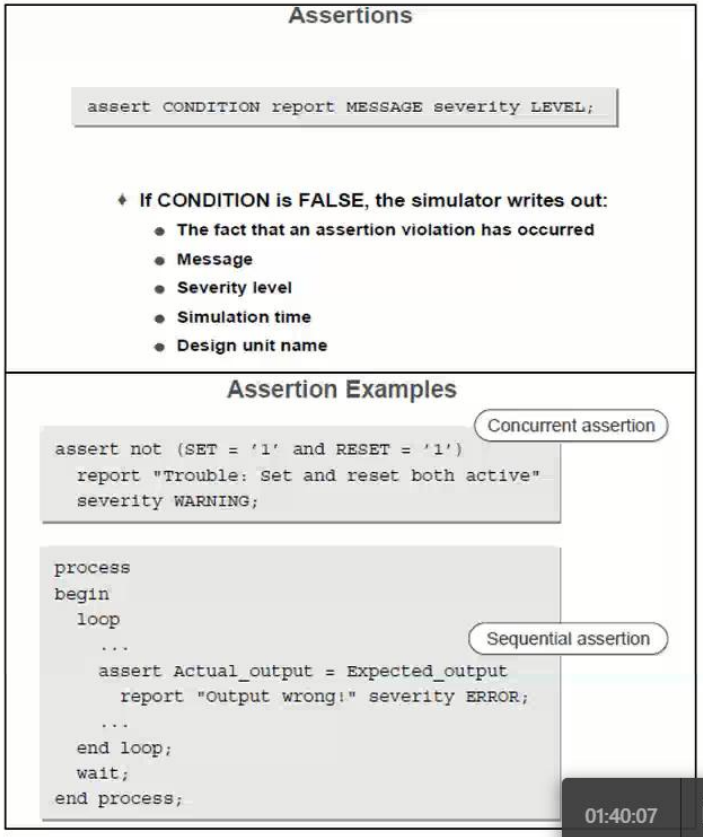
01:37:43



SIEMENS

Obeid, Mazen (DI SW ICS MNA RD QA QSIM 1)

# Assertions

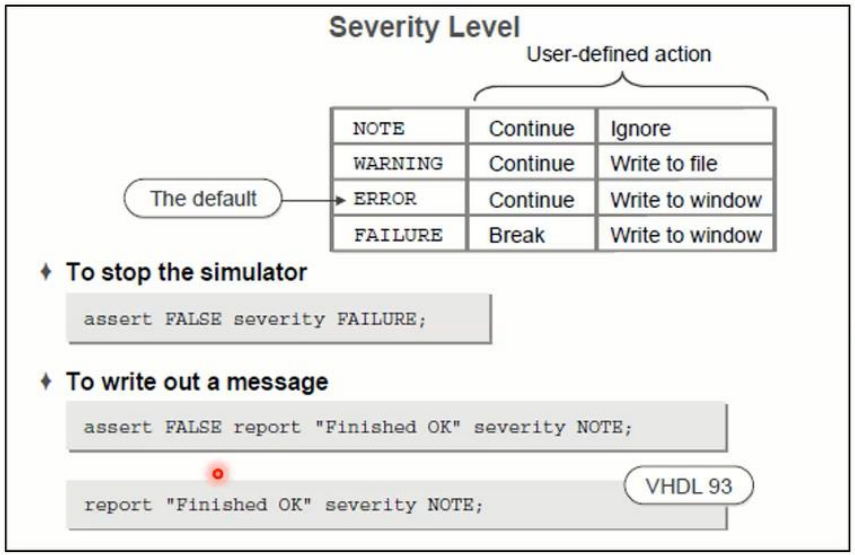



- Assertions are statements that can be used to check that any BOOLEAN condition is TRUE. If the assertion is not satisfied (the condition is FALSE), then a message is written out (usually to the simulator control window) including the string from the report part of the assert statement, the severity of the assertion, simulation time and the name of the VHDL design unit.
- It is important to note that an assertion does not change the behavior of a model, But is a passive construct that merely informs the user that some condition has been violated
- Assertions can be either concurrent or sequential statements, i.e.. they may be written outside or inside processes.



01:40:07
SIEMENS
Obeid, Mazen (DI SW ICS MNA RD QA QSIM 1)
MO

# Assertions Severity Levels




- The report and severity parts of the statement may be omitted.
- Severity defaults to *Error*



01:42:37

SIEMENS

# VHDL Subprograms : Procedures

**Procedures I**

- **Procedures are used to partition processes into a hierarchy**

```
signal A, B, C: BIT;
...
STIMULUS: process

  procedure ASSIGN (IP: BIT_VECTOR) is
  begin
    A <= IP(0);
    B <= IP(1);
    C <= IP(2);
  end ASSIGN;

begin
  ASSIGN("000");
  ...
  ASSIGN("010");
  ...
  ASSIGN("100");
  ...
end process STIMULUS;
```

- Just as design entities are used to partition a design into a hierarchical structure of interconnected blocks, so procedures are used to **partition up processes into smaller, hierarchical sequences of executable code**.
- A procedure consists of a **name, a set of parameters, a set of declarations, and a set of executable, sequential statements.**
- The example opposite shows a procedure ASSIGN used for the convenience in defining a set of stimuli.
- Each call to ASSIGN assigns values to the signals **A,B and C**. The **formal parameter** IP is replaced on each call of ASSIGN with an **actual parameter** and so can take different values each time.
- **Note that a procedure may read and write the values of signals and variables declared outside itself.**
- A procedure may contain *wait* statements. This will cause the calling process to suspend until the *wait's* condition is met.



SIEMENS

## VHDL Subprograms : Procedures

```
signal A, B, C: BIT;
signal P, Q, R: BIT;
...
STIMULUS: process

  procedure ASSIGN (IP: BIT_VECTOR;
                    signal X, Y, Z: out BIT;
                    PAUSE: TIME := 10 NS) is
  begin
    X <= IP(0);
    Y <= IP(1);
    Z <= IP(2);
    wait for PAUSE;
  end ASSIGN;

begin

  ASSIGN ("101", A, B, C, 5 NS);
  ASSIGN ("111", P, Q, R);
  wait;

end process STIMULUS;
```

- In the example opposite the procedure ASSIGN now has an additional three *signal* formal parameters.
- Note also a fourth additional parameter PAUSE has been added to control how long the calling process is suspended each time ASSIGN is called. This parameter has a default value, so the parameter is optional when the procedure is called.






SIEMENS

Obeid, Mazen (DI SW ICS MNA RD QA QSIM 1)

MO
Obeid, Mazen (DI SW ICS ...

# VHDL Subprograms : Procedures

**Parameters**

```
constant  }
variable  } N: {  in    }
signal    }     {  out   } T := default
                {  inout }
```

class  name  mode  type  default

```
procedure DEMO (A: Std_logic;          -- constant, in
                B: out Std_logic;      -- variable
                C: inout Std_logic;    -- variable
                signal D: Std_logic;   -- in
                signal E: out Std_logic);
...
signal Sig: Std_logic;
...
variable Var: Std_logic;
...
DEMO (Sig nand Var, Var, Var, Sig, Sig);
```

Default class and mode

- Procedure parameters may be of mode *in*, *out* or *inout*. **The default is *in***. *in* parameters can only be read. *out* parameters can only be updated. *inout* parameters can be read and updated.
- Parameter class may be constant, signal or variable. **in parameters default to constant. out and inout parameters default to variable.**
- In a procedure call, a signal parameter must be associated with a signal, a variable with a variable, but a constant parameter may be associated with any expression.
- A **return** statement may appear at any point in a procedure but is *optional* as there is an implied return after the last sequential statement within the procedure.

**Parameter Association**

```
procedure Write (L:         inout Line;
                 Value:     in Std_logic_vector;
                 Justified: in Side  := Right;
                 Field:     in Width := 0);
...
process
  variable Buff: Line;
begin
  Write (Buff, A, Left, 8);
  Write (Buff, B, Right);
  Write (Buff, C);
  Write (Buff);

  Write (L         => Buff,
         Value     => D,
         Justified => Left,
         Field     => 12);

  Write (Value => E,
         Field => 10,
         L     => Buff);
  ...
end process;
```

Default

Positional association

Illegal

Named association



SIEMENS

## VHDL Subprograms : Procedures

**RTL Example**

```
architecture RTL of Block1 is
  procedure Count (Load:  Std_logic;
                   Start, Stop: Unsigned;
                   signal Cnt: inout Unsigned) is
  begin
    if Load = '1' then
      Cnt <= Start;
    elsif Cnt < Stop then
      Cnt <= Cnt + 1;
    else
      Cnt <= Zero;
    end if;
  end procedure Count;
begin
  process (Clock, Reset) is
  begin
    if Reset = '1' then
      Acnt <= (others => '0');
    elsif Rising_edge (Clock) then
      Count (Aload, Astart, Astop, Acnt);
    end if;
  end process;
```

Synthesis: no clock edges, no wait statements

VHDL 93

- Here procedure “count” can be used inside as many process as I need without the need to replicate the code block.

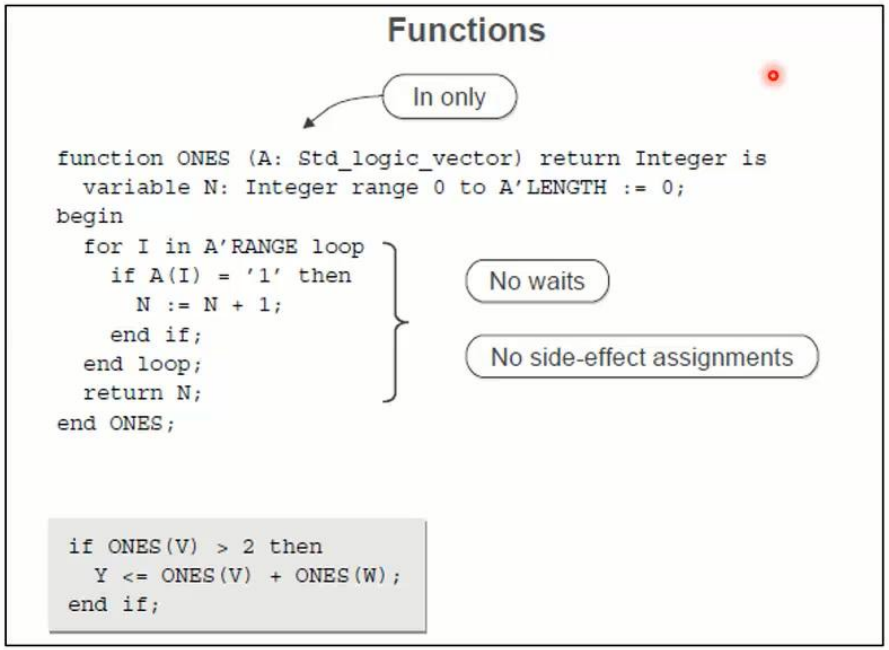





SIEMENS

Obeid, Mazen (DI SW ICS MNA RD QA QSIM 1)

# VHDL Subprograms : Functions




- ✓ VHDL functions can be used to define any logical or mathematical function which calculates a single result based on the values of a set of parameters
- ✓ A function **may contain any number of return statements**, each returning a value of the return type.
- ✓ **An error will occur if execution runs off the end of the function without finding a return statement**.
- ✓ Unlike a procedure a function cannot have side effects, i.e.. a function cannot assign a value to an object declared outside the function and cannot have *out* or *inout* parameters.
- ✓ A function cannot contain a *wait* statement. It would not make any sense for a function to suspend the process in the middle of evaluating the expression containing the function call.



SIEMENS

# VHDL Subprograms : Functions

**Parameters and Variables**

```
function PARITY (V: Std_logic_vector)
                return Std_logic is
  variable B: Std_logic := '0';
begin
  for I in V'RANGE loop
    B := B xor V(I);
  end loop;
  return B;
end PARITY;
```

Initialized every call

```
variable V4: Std_logic_vector(3 downto 0);
variable V8: Std_logic_vector(1 to 8);
```

```
P <= PARITY (V4);          -- V'RANGE = (3 downto 0)
P <= PARITY (V8);          -- V'RANGE = (1 to 8)
P <= PARITY ("01101");     -- V'RANGE = (0 to 4)
```

- Formal parameters of unconstrained array types are particularly useful, as the actual index constraint can be different each time the procedure is called. In the example opposite, the function PARITY is called three times with three different array lengths.

- Any declarations inside a subprogram are elaborated (i.e.. do their thing) every time the subprogram is called.
- In particular, variables and constants are initialized each time the subprogram is called. This is quite different from variables and constants declared inside a process, which are only initialized at time 0, and carry their old values forward between process executions.



SIEMENS

# Subprograms in Packages

Declaration

```
library IEEE;
use IEEE.Std_logic_1164.all;

package UTILITIES is

  function PARITY (V: Std_logic_vector)
                                  return Std_logic;

end UTILITIES;
```

Body

```
package body UTILITIES is

  function PARITY (V: Std_logic_vector)
                                  return Std_logic is
    variable B: Std_logic := '0';
  begin
    for I in V'RANGE loop
      B := B xor V(I);
    end loop;
    return B;
  end PARITY;

end UTILITIES;
```

- A subprogram may be declared in any of the places that we have been declaring signals, variables, and types. **However, the only really useful place to put a subprogram declaration is in a package declaration - otherwise its use will be restricted to one design unit.**
- There is a complication, however, in that the subprogram must be split into two parts.
  - The *subprogram declaration*, including the name, parameters and return type, goes in the package declaration.
  - The *subprogram body*, which repeats the subprogram declaration and adds the implementation of the subprogram, goes in the *package body*.

02:27:28



SIEM

MO

Obeid, Mazen (DI SW ICS ...

Obeid, Mazen (DI SW ICS MNA RD QA QSIM 1)

## VHDL-2008 : New Features

- Doulos documentation
  - Nice documentation for all vhdl-2008 constructs



SIEMENS

## VHDL-2008 : New Features

- **Hierarchical names**
  - Some of the new features in VHDL-2008 are intended for verification only, not for design. Verification engineers often want to write self-checking test environments. In VHDL this can be difficult as there is no easy way to access a signal or variable buried inside the design hierarchy from the top level of the verification environment.
  - VHDL-2008 addresses this by introducing ***external names***. An external name may refer to a (shared) variable, signal, or constant which is in another part of the design hierarchy. External names are embedded in double angle brackets << >>
  - Special characters may be used to move up the hierarchy ^ and to root the path in a package @ . Some examples:

```
<< signal .tb.uut.o_n : std_logic >>  -- hierarchical signal name
<< signal ^.^.a : std_logic >>        -- signal a two levels above
<< variable @lib.pack.v : bit >>      -- variable in a package pack
```






SIEM

Obeid, Mazen (DI SW ICS MNA RD QA QSIM 1)

MO
Obeid, Mazen (DI SW ICS ...

## VHDL-2008 : New Features

- **Enhanced Generics (Generic Types, subprograms and packages.)**
  - In VHDL generics are constants. In VHDL-2008 they may also be types, subprograms or packages.
  - Here is an entity that has a type generic (data_type) and a subprogram generic (function increment).

```
library ieee;
use ieee.std_logic_1164.all;
entity incrementer is
  generic (type data_type;
           function increment (x: data_type) return data_type);
  port (I : in data_type;
        O : out data_type;
        inc : in std_logic);
end entity incrementer;
architecture RTL of incrementer is
begin
  O <= increment(I) when inc = '1';
end architecture RTL;
```

This shows how we might instance this entity:

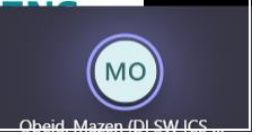
```
incr_inst : entity work.incrementer
  generic map ( data_type => std_logic_vector(7 downto 0),
```






SIEMENS





# VHDL-2008 : New Features

- **Enhanced Generics (Generic Types, subprograms and packages.)**
  - **Generic Package -** Support instantiation of the package as shown in the below example :

**Generic Package**

```
1  package InstPack is
2      generic(g1 : bit_vector(0 to 0));
3      impure function func(in1 : in bit_vector(0 to 0)) return bit_vector(0 to 0) ;
4  end package InstPack;
5
6  package body InstPack is
7      impure function func(in1 : in bit_vector(0 to 0)) return bit_vector(0 to 0) is
8      begin
9      return in1;
10     end function func;
11 end package body InstPack;
12
13 --This package could be instantiated as :
14 package InstPack1 is new work.InstPack generic map(g1 => bit_vector'("0"));
```






SIEMENS

## VHDL-2008 : New Features

- **UVVM**
  - UVVM is a free and open-source methodology for making structured VHDL-based testbenches inspired from UVM ( Universal Verification methodology that based on SystemVerilog ) but **it is component based not oop (class) based like UVM.**
  - UVVM consists of utility library and VVC (VHDL Verification Component) framework including this utility library.
  - UVVM supports OSVVM library for Random generation and Functional Coverage.
  - UVVM is based on commands that are sent from the sequencer to the VVC which is talking to the DUT.
  - UVVM GitHub Reference

  - **What is OSVVM?**
    - OSVVM is an intelligent testbench methodology that allows mixing of "Intelligent Coverage" with directed, algorithmic, file based, and constrained random approaches. OSVVM is an integrated environment designated for verification of VHDL.
    - OSVVM stands for "Open Source VHDL Verification Methodology". OSVVM is a set of VHDL packages, initially developed by Aldec and Synthworks. OSVVM helps you adopt modern constrained random verification techniques using VHDL.
    - Reference






SIEMENS